

Hands-On L3: Installing Docker & Building a Multi-Container Microservice

This hands-on will help you install and set up Docker Desktop, deploy a Python Flask web application with a Redis cache, and run a PostgreSQL database. You'll practice container networking, multi-container orchestration with Docker Compose, and submit your results via GitHub.

This hands-on is worth 1 point. Submit the repository link on Canvas by 11:59 pm on the day of the class. Expect it to take 60-75 minutes.

Learning Goals

- Understand the fundamentals of working with containerized environments.
- Gain experience in setting up and configuring services in a cloud or local development environment.
- Learn how to connect and orchestrate multiple components in a system.
- Develop skills in testing, monitoring, and documenting deployed applications.

What You'll Do:

- Install and verify Docker Desktop.
- Run a PostgreSQL container.
- Set up Flask app and Redis cache files.
- Build and connect services with Docker Compose.
- Test in the browser and capture screenshots of output, as well as Docker Desktop showing the containers.
- Push the project to GitHub.
- Create an Issue for the errors faced during hands-on.
 - **Note: Some errors are introduced intentionally in the code. Log these errors under the Issues tab in GitHub.**

1. Install Docker Desktop

Download Docker Desktop from <https://docs.docker.com/get-started/get-docker/>. That page covers Windows, macOS, and Linux. Docker Desktop is free for personal and educational use.

Windows

1. Go to [Docker Desktop for Windows:](https://docs.docker.com/desktop/setup/install/windows-install/)
<https://docs.docker.com/desktop/setup/install/windows-install/>
2. Download Docker Desktop Installer.exe.
3. Run it with default settings.
4. Start Docker Desktop from the Start Menu.
5. If prompted, enable **WSL 2** and restart.

macOS

1. Go to [Docker Desktop for Mac:](https://docs.docker.com/desktop/setup/install/mac-install/)
<https://docs.docker.com/desktop/setup/install/mac-install/>
2. Download the installer for your chip type (**Apple Silicon** or **Intel**).
3. Open the .dmg file and drag **Docker.app** into **Applications**.
4. Open **Docker.app** and follow security prompts.

Linux: follow the Docker Desktop for Linux instructions on the same page, or install Docker Engine plus the Compose plugin from your distribution's packages. Everything in this hands-on works with either.

Verify Installation

Open **Command Prompt** (Windows) or **Terminal** (Mac) and run:

```
docker --version
```

2. PostgreSQL Setup with Docker

- a. Pull the PostgreSQL image

```
docker pull postgres:16
```

- b. Create and start a PostgreSQL instance

```
docker run -d -p 5432:5432 --name postgres1 -e POSTGRES_PASSWORD=pass12345 postgres:16
```

- c. Open a terminal to the container

```
docker exec -it postgres1 bash
```

- d. Interact with PostgreSQL using psql

```
psql -d postgres -U postgres
```

Type `\q` to leave `psql`, then `exit` to leave the container. When you have finished this section, stop and remove the container so it does not hold port 5432:

```
docker stop postgres1 && docker rm postgres1
```

If port 5432 is already in use (for example, you have PostgreSQL installed locally), publish a different host port instead, such as `-p 5433:5432`.

3. Build a Python Web App with Docker Compose in code environments (e.g., VS Code)

- a. Define application dependencies in `requirements.txt`

```
flask==3.0.3  
redis==5.0.7
```

- b. Define the application in `app.py`

```
import time  
  
import redis  
from flask import Flask  
  
app = Flask(__name__)  
cache = redis.Redis(host='redis', port=6379)  
  
def get_hit_count():  
    retries = 5  
    while True:  
        try:  
            return cache.incr('hits')  
        except redis.exceptions.ConnectionError as exc:  
            if retries == 0:  
                raise exc  
            retries -= 1  
            time.sleep(0.5)  
  
@app.route('/')  
def hello():  
    count = get_hit_count()  
    return 'Hello World! I have been seen {}  
        times.\n'.format(count)
```

Indentation matters in Python. Retype the code rather than pasting it from this document if the spacing does not survive the copy.

- c. Create a Dockerfile to containerize the application. Use the following template:

```
FROM python:3.12-alpine
WORKDIR /code
ENV FLASK_APP=app.py
ENV FLASK_RUN_HOST=0.0.0
RUN apk add --no-cache gcc musl-dev linux-headers
COPY requirements.txt requirements.txt
RUN pip install -r requirements.txt
EXPOSE 5000
COPY . .
CMD ["flask", "run"]
```

- d. Define services in a `compose.yaml` file

```
services:
  web:
    build: .
    ports:
      - "8000:5000"
    depends_on:
      - redis
  redis:
    image: "redis:alpine"
```

- e. Build and run the application

```
docker compose up
```

- f. Open the application in a browser at:

<http://localhost:8000>

- g. Refresh the page a few times; the counter increases because Redis is keeping the count. To stop the stack, press Ctrl+C in the terminal and then run:

```
docker compose down
```

4. Submission

Expected submission format: One-page report with screenshots from the Docker Desktop app showing the containers' logs and the GitHub repo link in the document.

- Upload all files to GitHub (app.py, Dockerfile, compose.yaml, requirements.txt, Readme.md, and a Document with screenshots).
- Readme.md: This file should contain the execution steps (**note: ensure to Quote the code. This can be found in further readings below.**) and details as to what you have learned in this hands-on with appropriate formatting.

- c. Log the errors encountered in the hands-on exercises (**you will face errors**) under the Issues tab in GitHub.
- d. Upload a document in the repository with screenshots of:
 - i. Docker Desktop app showing the containers.
 - ii. Output obtained after the hands-on.
- e. Submit the link to the GitHub repository in Canvas.

Repository requirements

Create the repository as **Public** (don't add the instructor and the TAs as collaborators). Add a .gitignore containing at least __pycache__ / so that build artefacts stay out of the repository. Name the readme README.md.

Issues. Open **one issue per distinct error** you hit. Each issue needs a title naming the error, a description with what you expected, what actually happened, and the exact message, the bug label, and it should be assigned to you. Say what fixed it, then close the issue.

5. Further Readings:

Docker CLI Cheat Sheet:

https://docs.docker.com/get-started/docker_cheatsheet.pdf

Basic writing and formatting syntax for Readme:

<https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>